

# CHANGE ASSURANCE RUNTIME

## Implementation-Ready Project Proposal and Technical Architecture

*Native execution. Scoped agent authority. Trusted tools. Environment provenance. Replayable change lifecycles.*

**Working thesis: Every human- or AI-generated software change should be attributable, bounded, independently observed, verifiable against intent, reproducible enough to debug, and recoverable wherever recovery is technically possible.**

| Document         | Value                                                                                                   |
|------------------|---------------------------------------------------------------------------------------------------------|
| Purpose          | Technical proposal and build specification                                                              |
| Primary audience | Founders, engineers, security/platform engineers, contributors                                          |
| Initial platform | Windows-first native developer runtime, with Linux/macOS adapters later                                 |
| Primary users    | Developers using Codex, Claude Code, Cursor, Gemini CLI, terminals, CI systems, and other coding agents |
| Project stage    | Architecture and implementation planning                                                                |
| Date             | September 2026                                                                                          |

# Executive Summary

AI coding agents can now modify repositories, run arbitrary commands, install packages, call external tools, use credentials, open pull requests, and increasingly participate in deployment workflows. Existing safety approaches tend to choose between unrestricted execution and repeated human approval. Both approaches break down as agent autonomy increases.

This project introduces a native Change Assurance Runtime. Rather than treating an AI agent session as the primary object, it treats a software Change as a persistent lifecycle that begins with intent, spans local execution and verification, and can later connect to Git, CI, artifacts, release, and production outcomes. The runtime is agent-neutral and should work with existing coding agents instead of replacing them.

**The core product is not "undo for AI agents." It is an execution and assurance layer for software development, with rollback as one mechanism inside a broader model of causality, authority, trust, verification, and replay.**

## Selected strategic pillars

| Pillar                                | Purpose                                                                                                                                   |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Change Assurance Runtime           | Create and manage the full lifecycle of a software change. Capture effects, build evidence, understand blast radius, and expose recovery. |
| 2. Agent Identity + Credential Broker | Give agents and tools scoped, time-bound authority without exposing durable user credentials inside agent processes.                      |
| 3. Replay / Agent Debugger            | Provide time-travel inspection of agent execution and progressively stronger replay modes for debugging failures.                         |
| 4. Agent Tool Supply Chain            | Track provenance, permissions, signatures, versions, and trust of MCP servers, skills, plugins, CLI tools, and other agent capabilities.  |
| 5. Native Environment Provenance      | Track relevant host-machine state and changes so Docker is not required to understand environment drift or recover host-side mutations.   |

## Primary product promise

- Know what changed - not only repository diffs, but processes, dependencies, tools, credentials, and environment effects.
- Know who had authority - map human identity to agent identity, process identity, capability, scope, and expiration.
- Know what was trusted - establish provenance and permissions for agent-facing tools and external capabilities.
- Know what happened to the real machine - detect changes that Git cannot represent.
- Know why something failed - reconstruct and replay the causal sequence instead of relying only on chat logs.
- Recover with evidence - verify rollback rather than claiming that an undo succeeded.

# 1. Problem Definition

## 1.1 The autonomy dilemma

Coding agents derive value from acting, not merely talking. They need access to files, shells, package managers, developer tools, source control, test systems, APIs, and eventually cloud/deployment infrastructure. Current control mechanisms generally create two bad modes:

- Unrestricted mode: high productivity, but failures are discovered after state has already changed.
- Approval-everywhere mode: safe in theory, but repeated prompts create approval fatigue and users stop evaluating them meaningfully.

A better system should allow low-risk, reversible work to proceed quickly while enforcing stronger controls around actions whose effects cannot be safely contained, understood, or reversed.

## 1.2 Why Git is insufficient

| What Git sees               | What Git does not see                               |
|-----------------------------|-----------------------------------------------------|
| Tracked source file changes | Global package installs                             |
| Commits and branches        | Processes and background servers                    |
| Repository history          | PATH, registry, services, certificates, SDK changes |
| Merge conflicts             | Credential use and delegated authority              |
| Repository diff             | MCP/tool provenance and runtime permissions         |
| Committed content           | External API effects and information exposure       |
| Branch state                | Agent reasoning path and tool responses             |

## 1.3 Why snapshots are insufficient

Whole-machine or whole-filesystem snapshots are useful implementation primitives, but they cannot be the product abstraction. A developer, sync client, IDE, or another agent may modify state after the tracked agent did. Blindly restoring an old snapshot can erase legitimate later work. The system therefore needs causal write sets and conflict-aware rollback.

```
For each mutable resource:
  BEFORE   = state when Change first touches the resource
  PRODUCED = state after the Change modifies it
  CURRENT  = state when recovery is requested

If CURRENT == PRODUCED:
  recovery can usually restore BEFORE safely
Else:
  mark recovery conflict and require merge / selective resolution
```

## 1.4 Why technically reversible is not socially reversible

Deleting a sent message does not erase the fact that a recipient may have read it. Refunding a charge does not remove all side effects of the charge. Reverting a production deployment does not erase downtime. The runtime therefore needs a richer effect model than a binary reversible/irreversible flag.

## 2. Product Principles

1. The Change is the root object. Agent sessions, human edits, processes, tests, PRs, artifacts, and recovery operations attach to a Change.
2. Observation beats narration. The agent may propose or describe state, but the runtime independently records authoritative evidence.
3. Native first. Developers should not need to containerize their existing repository or configure Docker to receive core benefits.
4. Least authority by construction. Agents should request capabilities rather than inherit durable human credentials.
5. Reversibility is typed. Exact rollback, conditional rollback, compensation, staging, irreversibility, and unknown semantics are distinct.
6. No safety theater. A large immutable log is not useful if a human cannot understand what matters. The UI must compress evidence by risk and causality.
7. Existing standards should be composed, not reinvented. Git, SLSA-style provenance, OpenTelemetry, workload identity patterns, package metadata, and CI systems should feed the Change model.
8. Agent neutral. Codex, Claude Code, Cursor, Gemini CLI, terminals, and future agents should operate through the same underlying runtime.

### 3. Overall Architecture

**Architectural rule: the system may contain many services, but every user-visible outcome should be explainable in terms of a Change, its Effects, its Evidence, its Authority, and its Recovery state.**



#### 3.1 Runtime components

| Component            | Responsibility                                                            | Initial form                                   |
|----------------------|---------------------------------------------------------------------------|------------------------------------------------|
| Change Service       | Own Change state machine, contracts, metadata, lifecycle, and API.        | Local daemon service                           |
| Event/Effect Journal | Append-only records of actions and state transitions.                     | SQLite + immutable event rows + content hashes |
| Native Executor      | Launch and supervise agent/tool processes under OS-specific restrictions. | Windows-first adapter                          |

|                     |                                                                                  |                                                           |
|---------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------|
| Process Supervisor  | Track child processes, resource use, termination, orphan cleanup.                | Windows Job Objects initially                             |
| Filesystem Tracker  | Capture created/modified/deleted paths and before/after identity.                | User-mode journal first, optional deeper OS adapter later |
| Environment Tracker | Capture selected host state and persistent mutations outside repository.         | Snapshot/fingerprint collectors                           |
| Credential Broker   | Mint or proxy scoped credentials; keep durable secrets out of agent process.     | GitHub first; generic capability API                      |
| Tool Registry       | Inventory tools/MCP/skills, signatures, versions, capabilities, trust status.    | Local registry + signed manifests                         |
| Assurance Engine    | Choose and execute checks relevant to the Change.                                | Test/build/lint adapters + graph heuristics               |
| Recovery Engine     | Restore exact effects, detect conflicts, run compensations, verify result.       | File/process/environment first                            |
| Replay Engine       | Store timeline, captured responses, and reconstructable execution state.         | Trace replay first                                        |
| Desktop/CLI         | Expose Change list, inspection, review, recovery, tool trust, environment drift. | CLI first, desktop after data model stabilizes            |

## 4. Core Data Model

### 4.1 Change

```
Change {
  id
  title
  intent_text
  contract_version
  repository_id
  status
  risk_class
  created_by_human_id
  active_agent_ids[]
  environment_baseline_id
  started_at
  completed_at
  assurance_state
  recovery_state
}
```

### 4.2 Effect

```
Effect {
  id
  change_id
  parent_effect_id?
  actor_id
  process_id?
  type                // filesystem.write, dependency.install, tool.call, ...
  target              // canonical resource identity
  before_digest?
  produced_digest?
  current_digest?
  restoration_class    // exact, conditional, compensating, stageable, none, unknown
  exposure_class       // private, team, external, public
  verification_state
  occurred_at
}
```

### 4.3 Actor and authority

```
Actor {
  id
  kind                // human, agent, process, tool, service
  parent_actor_id?
  identity_provider?
  executable_digest?
}

Delegation {
  human_actor_id
  delegated_actor_id
}
```

```

change_id
capability
resource_scope
issued_at
expires_at
credential_reference? // never raw secret material
}

```

## 4.4 Tool record

```

ToolRecord {
  tool_id
  name
  version
  publisher
  source
  artifact_digest
  signature_state
  capabilities[]
  filesystem_scope[]
  network_scope[]
  credential_requirements[]
  trust_state // approved, provisional, denied, unknown
  first_seen_at
  last_seen_at
}

```

## 4.5 Environment passport

```

EnvironmentPassport {
  id
  os_build
  architecture
  toolchain_versions{}
  path_digest
  environment_key_digests{}
  global_package_sets{}
  relevant_service_state{}
  sdk_state{}
  git_configuration_digest
  captured_at
}

```

# 5. Change State Machine

```

DRAFT
-> ACTIVE
  -> LOCALLY_VERIFIED
    -> REVIEW_READY
      -> PR_OPEN
        -> CI_VERIFIED

```

- > ARTIFACT\_BUILT
- > DEPLOYED
- > OBSERVING
- > STABLE

From most states:

- > PAUSED
- > BLOCKED
- > FAILED
- > RECOVERY\_PENDING
- > RECOVERING
- > RECOVERED\_VERIFIED
- > RECOVERY\_CONFLICT
- > RECOVERY\_FAILED

The initial implementation does not need every late-stage state. G1 should implement DRAFT, ACTIVE, LOCALLY\_VERIFIED, REVIEW\_READY, RECOVERY\_PENDING, RECOVERING, RECOVERED\_VERIFIED, RECOVERY\_CONFLICT, and FAILED. CI/deployment states arrive later without redesigning the root model.

## 6. End-to-End User Flow

### 6.1 First install

1. Install desktop/CLI runtime. The installer registers the local daemon and native execution helper.
2. Select or open an existing repository. No Dockerfile or repository conversion is required.
3. Runtime discovers Git state, project language/toolchain, package managers, likely test/build commands, existing CI configuration, and environment baseline.
4. Optionally connect GitHub or another provider. Durable credentials remain in the broker/OS secret store, not inside agent child processes.
5. Runtime builds an initial project/SDLC graph incrementally.

### 6.2 Starting a Change

```
> change run codex
```

User: Refactor the auth module to use rotating refresh tokens.

Runtime creates:

C-1048 Rotating refresh tokens

Risk domains: authentication, token lifecycle

Expected scope: src/auth, tests/auth

Forbidden by default: production mutation, force push, unrestricted secret access

The Change Contract can be lightweight. The runtime proposes it automatically and only requires confirmation when scope or risk is unusual.

### 6.3 During execution

- Local reversible file edits proceed without repeated approval.
- Process tree is supervised and attributed to the Change.
- Commands and exit codes are independently recorded.
- Dependency modifications become typed effects, not generic file diffs.
- Tool/MCP usage is resolved to Tool Records and checked against trust/capability metadata.
- Credential use is brokered and attributed to a capability and resource scope.
- Persistent machine drift is attributed to the Change where possible.
- Scope deviations trigger focused review rather than generic command prompts.

### 6.4 Change review

AUTH TOKEN ROTATION - C-1048

Intent coverage 6/6

Source files changed 17

Tests changed 3

Dependency changes 1

Persistent host drift 0

Unknown tool usage 0

Credential scopes used github:repo-read only

Assurance

Unit tests PASS

Integration tests PASS

|                   |                                    |
|-------------------|------------------------------------|
| Build             | PASS                               |
| API compatibility | PASS                               |
| Missing evidence  | concurrent token rotation scenario |

#### Recovery

|                  |       |
|------------------|-------|
| Files            | exact |
| Processes        | exact |
| Dependency state | exact |
| External effects | none  |

## 6.5 Recovery

If the user selects Undo, the engine does not blindly restore a snapshot. It computes the Change write-set, checks current resource versions, identifies conflicts, executes supported restoration actions, and verifies post-conditions.

#### Recovery result

17/17 files restored and verified  
3/3 child processes terminated  
dependency lock restored  
host drift remaining: 0  
conflicts: 0

Final state: RECOVERED\_VERIFIED

## 7. Pillar 1 - Change Assurance Runtime

### 7.1 Responsibilities

- Own Change identity and lifecycle.
- Capture causal relationships among agent sessions, human edits, processes, commands, files, dependencies, tests, tool calls, and external effects.
- Maintain the authoritative Effect Journal.
- Select assurance work relevant to the observed impact surface.
- Expose recovery readiness and unresolved conflicts.
- Produce a Change Passport for review and future historical analysis.

### 7.2 Assurance model

Avoid treating an assurance percentage as a magical probability of correctness. Internally, model assurance as evidence coverage. The UI may summarize evidence, but should preserve why a requirement is verified, unverified, conflicting, or not applicable.

| Evidence domain    | Example evidence                                                |
|--------------------|-----------------------------------------------------------------|
| Intent             | Acceptance criteria mapped to observed result                   |
| Tests              | Relevant tests selected and their authoritative reporter output |
| Build              | Compiler/build command, artifact hash, exit result              |
| Static correctness | Types, lint, schema validation                                  |
| Security           | Relevant scanner findings and policy checks                     |
| Dependencies       | Version change, lockfile, source, known advisories, trust state |
| Environment        | No unexplained persistent drift                                 |
| Recovery           | Known recovery path and verification capability                 |

## 8. Pillar 2 - Agent Identity and Credential Broker

### 8.1 Threat model

- Agent process receives a long-lived secret and exfiltrates it.
- Untrusted tool/MCP server inherits environment credentials.
- Agent uses a credential outside the intended repository or task.
- A later process reuses a credential after the Change has ended.
- Audit trail cannot distinguish user intent from agent action or tool action.

### 8.2 Authority chain

Human identity

- > Change authorization
- > Agent actor
  - > Tool/process actor
    - > Capability request
    - > Credential broker
      - > narrow external authority

### 8.3 Initial broker scope

Do not start with AWS, Slack, Gmail, and every SaaS provider. Implement GitHub first because it is tightly aligned with SDLC and offers a clear set of repository-scoped operations. Store durable provider credentials using the platform secret store, issue/proxy narrower operations through the broker, and journal every use.

| G1 capability      | Behavior                                               |
|--------------------|--------------------------------------------------------|
| github.repo.read   | Allow read-only API requests for configured repository |
| github.branch.push | Optional branch-specific write capability              |
| github.pr.create   | Permit PR creation for Change branch                   |
| github.force_push  | Denied by default                                      |
| github.secret.read | Denied; raw secret retrieval should not be needed      |

## 9. Pillar 3 - Replay and Agent Debugger

### 9.1 Replay levels

| Level                       | Description                                                                | G1?   |
|-----------------------------|----------------------------------------------------------------------------|-------|
| Trace replay                | Reconstruct the timeline exactly from recorded events and evidence.        | Yes   |
| Tool-response replay        | Feed recorded tool responses to a debugging harness.                       | G2    |
| Filesystem replay           | Recreate the relevant repository/environment state at a point in time.     | G2    |
| Controlled execution replay | Re-run from a checkpoint with fixed recorded external inputs.              | G3    |
| Forked replay               | Replay from a chosen point with a different model, instruction, or policy. | G3/G4 |

### 9.2 Replay event requirements

- Monotonic sequence number and timestamp.
- Change ID, actor ID, process ID, parent event/effect IDs.
- Command/tool invocation identity and arguments with secret redaction.
- Exit code / structured result digest.
- Before/after resource digests where applicable.
- Captured environment revision.
- Model/provider reference and request/response hashes when available.
- Policy decision and capability decision.

### 9.3 Debugging UX

#### Timeline

```
13:41:02 READ src/auth/token.ts
13:41:08 WRITE src/auth/token.ts
13:41:12 RUN npm test
13:41:31 TEST FAIL 14 cases
13:41:34 AGENT decision boundary
13:41:40 RUN package upgrade
13:41:59 WRITE package-lock.json
13:42:03 TEST PASS
```

Select any row -> inspect input state, actor, authority, output, causal children, and recovery implications.

## 10. Pillar 4 - Agent Tool Supply Chain

### 10.1 What counts as a tool

- MCP servers and connectors.
- Agent skills/plugins/extensions.
- CLI programs launched by an agent.

- Language-specific helper packages that execute code during install/build.
- Local scripts fetched or generated during an agent session.

## 10.2 Tool manifest

```
tool: github-mcp
version: 2.7.1
publisher: GitHub
digest: sha256:...
signature: valid
capabilities:
  - github.repo.read
  - github.pr.create
network:
  - api.github.com
filesystem: []
credentials:
  - github scoped capability
trust: approved
```

## 10.3 Trust lifecycle

```
UNKNOWN -> OBSERVED -> PROVISIONAL -> APPROVED
      \-> DENIED
```

Any version, publisher, signature, digest, or capability change can invalidate prior trust.

## 11. Pillar 5 - Native Environment Provenance

### 11.1 Why it matters

A Docker-free developer experience means the real host is part of the software-development system. Repository rollback alone is insufficient if an agent globally installs packages, modifies PATH, creates a service, changes configuration, or alters SDK state. The runtime must therefore fingerprint relevant machine state and attribute persistent drift.

### 11.2 G1 environment collectors

| Category              | Windows-first collector                                                               |
|-----------------------|---------------------------------------------------------------------------------------|
| OS                    | Version/build, architecture, selected security/runtime properties                     |
| Languages             | node/npm, Python/pip, Java, .NET, Rust, Go when detected                              |
| Git                   | Version, selected config, hooks metadata                                              |
| PATH                  | Canonical entries + digest                                                            |
| Package managers      | Relevant global package sets where query cost is reasonable                           |
| Services              | Only services/processes created or materially touched during Change when attributable |
| Environment variables | Names + salted hashes for sensitive values; never persist raw secrets by default      |
| SDK/toolchain         | Detected compiler/build tool versions                                                 |

### 11.3 Attribution strategy

Do not attempt to constantly diff the entire operating system. Use a layered approach: baseline fingerprint + event-driven observations from supervised processes + targeted re-snapshot of relevant surfaces at Change completion. This keeps overhead bounded while catching the host mutations most likely to matter.

## 12. Native OS Execution Architecture

**Cross-platform parity is a later goal. The architecture should define a common execution contract but allow different enforcement techniques per OS.**

| OS      | Initial enforcement direction                                                                        | Important primitive                                                                     |
|---------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Windows | Restricted execution helper + supervised child tree + filesystem/process tracking + brokered secrets | Job Objects; restricted tokens/AppContainer where appropriate; Windows credential store |
| Linux   | Namespace/LSM-aware supervised execution without requiring Docker UX                                 | Landlock, cgroups, procfs, optional namespaces                                          |
| macOS   | Native process supervision and event observation                                                     | Endpoint Security and platform sandboxing capabilities                                  |

## 12.1 Windows G1 process model

Desktop / CLI

|

v

Local Runtime Daemon

|

+--> Privileged setup/helper (minimal surface)

|

+--> Change Executor

|

+--> Windows Job Object J-C1048

|

+--> Agent process

+--> child shells

+--> compilers/test runners

+--> local dev servers

All child PIDs are mapped back to C-1048.

## 13. Initial API Surface

The local daemon should expose a versioned localhost API. Authentication should use a local application principal rather than relying on an unauthenticated port.

| Method | Path                              | Purpose                                |
|--------|-----------------------------------|----------------------------------------|
| POST   | /v1/changes                       | Create Change and contract             |
| GET    | /v1/changes                       | List current/history Changes           |
| GET    | /v1/changes/{id}                  | Get Change summary                     |
| POST   | /v1/changes/{id}/start            | Start supervised execution             |
| POST   | /v1/changes/{id}/pause            | Pause or stop Change executor          |
| GET    | /v1/changes/{id}/effects          | Paginated Effect Journal               |
| GET    | /v1/changes/{id}/timeline         | Replay timeline                        |
| GET    | /v1/changes/{id}/assurance        | Evidence/verification state            |
| POST   | /v1/changes/{id}/recovery/plan    | Compute recovery plan                  |
| POST   | /v1/changes/{id}/recovery/execute | Execute approved recovery plan         |
| GET    | /v1/tools                         | Tool registry                          |
| POST   | /v1/tools/{id}/trust              | Approve/deny tool version/capabilities |
| GET    | /v1/environment/current           | Current environment passport           |
| GET    | /v1/identity/delegations          | Active delegated capabilities          |

## 14. Persistence and Storage

### 14.1 SQLite-first authority

A local SQLite database is appropriate for the initial authority store: transactional, portable, debuggable, WAL-capable, and already familiar from CML. Large immutable blobs, logs, or before-images can live in a content-addressed blob store referenced by the database.

| Table                 | Core responsibility                                        |
|-----------------------|------------------------------------------------------------|
| changes               | Change identity, lifecycle, risk, contract linkage         |
| change_contracts      | Versioned intent/constraint/acceptance contract            |
| actors                | Human, agent, process, tool, service identities            |
| delegations           | Scoped capability grants and expiry                        |
| processes             | Supervised process tree                                    |
| effects               | Semantic action journal                                    |
| resource_versions     | Before/produced/current digests and canonical resource IDs |
| environment_passports | Environment revisions                                      |
| tools                 | Tool identities and trust metadata                         |
| tool_capabilities     | Declared/observed capability set                           |
| assurance_checks      | Checks, evidence, results, missing coverage                |
| recovery_plans        | Ordered recovery actions and conflict state                |

|               |                                     |
|---------------|-------------------------------------|
| replay_events | Time-ordered debugger timeline      |
| artifacts     | Content-addressed blobs/log bundles |

## 14.2 Integrity

- Each journal row receives a monotonically increasing sequence within a Change.
- Sensitive payloads are redacted/encrypted; the journal records stable digests rather than raw secrets.
- Optionally chain journal hashes to make local tampering detectable.
- Store before-images only where required for recovery; use content addressing for deduplication.

## 15. Proposed Repository Layout

```
change-runtime/  
  apps/  
    desktop/      # Electron/Tauri/React later  
    cli/          # change CLI  
  
  runtime/  
    daemon/       # local API + lifecycle orchestration  
    executor/     # common execution interface  
    recovery/  
    assurance/  
    replay/  
    identity/  
    tools/  
    environment/  
  
  platform/  
    windows/      # Windows native helper/backend  
    linux/        # later  
    macos/        # later  
  
  adapters/  
    agents/  
      codex/  
      claude_code/  
      generic_shell/  
    scm/  
      git/  
      github/  
    package_managers/  
      npm/  
      pip/  
    tests/  
      pytest/  
      jest/  
      generic_exitcode/  
  
  schemas/  
    change/  
    effect/  
    tool_manifest/  
    replay/  
  
  tests/  
    unit/  
    integration/  
    fixtures/  
    destructive_sandbox/  
  
  docs/  
    architecture/  
    threat-model/  
    adapters/
```

## 15.1 Language recommendation

A pragmatic implementation can use Python for the orchestration daemon and fast iteration, with a small Rust or C/C++ native helper for security-sensitive Windows process/OS operations once the boundaries stabilize. Do not begin with a large native rewrite. Keep privileged code as small as possible. A Rust core may later become attractive for portability and strong typing around journal/recovery semantics.

## 16. Build Plan

**The build should produce useful vertical slices early. Do not start by implementing every policy, provider, or OS. The first compelling demo is: run an existing agent on a real repository, observe all local effects, explain them, and reliably undo them.**

| Phase              | Goal                                                       | Exit criterion                                                           |
|--------------------|------------------------------------------------------------|--------------------------------------------------------------------------|
| P0 - Foundation    | Schemas, daemon, CLI, Change lifecycle, event journal      | Can create/inspect a Change and append validated events                  |
| P1 - Local Effects | Process tree + file/command effects + Git awareness        | Agent session produces a causal local timeline                           |
| P2 - Recovery      | Before-images, write-set recovery, conflicts, verification | Demo exact local undo without deleting unrelated edits                   |
| P3 - Environment   | Environment passport + attributable host drift             | Global/toolchain drift is visible and recoverable for supported surfaces |
| P4 - Assurance     | Test/build discovery, relevant evidence, Change review     | Runtime can explain what was verified and what remains unknown           |
| P5 - Identity      | Actor tree + GitHub credential broker + delegation         | Agent can create scoped GitHub action without raw durable credential     |
| P6 - Tool Trust    | Tool manifest/registry + capability checks                 | Unknown high-capability MCP/tool produces meaningful risk decision       |
| P7 - Replay        | Timeline UI + trace replay + state inspection              | Developer can pinpoint and inspect the first bad effect                  |
| P8 - GitHub/CI     | PR and CI continuity into same Change                      | Change Passport spans local execution to CI result                       |

# 17. Detailed Implementation Milestones

## P0 - Foundation

- Define JSON/Pydantic schemas for Change, Actor, Effect, ResourceVersion, EnvironmentPassport, AssuranceCheck, RecoveryPlan, ReplayEvent, ToolRecord.
- Implement SQLite schema/migrations with WAL and foreign keys.
- Implement Change state machine with explicit transition validation.
- Implement append-only journal API and sequence allocation.
- Implement local CLI: change new, change list, change show, change events.
- Add crash-safe daemon startup and interrupted Change recovery state.

## P1 - Local Effects

- Implement Windows Change Executor that starts a command/agent under a supervised Job Object.
- Capture process tree creation/termination and command metadata.
- Track repository filesystem changes. Initial method may combine pre/post snapshots with process-attributed hints; deeper hooks can follow.
- Canonicalize paths robustly, including Windows short/long path aliases.
- Record Git HEAD, working tree baseline, branch, index state, and later state.
- Adapter wrappers: Codex, Claude Code, generic shell.

## P2 - Recovery

- Capture before-images of files at first write/delete within a Change.
- Store content-addressed before/produced digests.
- Build recovery planner that checks CURRENT vs PRODUCED and classifies conflicts.
- Restore safe paths, remove Change-created paths, restore deleted paths.
- Terminate Change-owned process trees.
- Verify every restored path by digest.
- Never overwrite a post-Change human edit silently.

## P3 - Environment Provenance

- Capture baseline OS/toolchain environment.
- Track PATH, detected global package sets, selected config surfaces.
- Attribute changes initiated by supervised process tree where feasible.
- At Change completion, compare relevant surfaces and report persistent drift.
- Implement recovery adapters only for well-understood environment effects; unknown effects remain visible instead of faking reversibility.

## P4 - Assurance

- Discover package manager and project scripts.
- Implement pytest/Jest/generic command adapters.
- Normalize structured test results where possible.
- Build change-to-test heuristics using paths, project config, import relationships, and historical runs.
- Show evidence coverage, not an unexplained safety probability.
- Add contract deviation detection: unexpected directories, dependency major changes, migration files, security-sensitive paths.

## P5 - Identity and GitHub Broker

- Represent human -> Change -> agent -> process/tool authority chain.

- Keep provider credential in OS-protected storage.
- Expose narrow GitHub operations through brokered local API.
- Support repository/branch-scoped capability grants with expiry.
- Audit every request, provider response metadata, and actor/effect relationship.

## **P6 - Tool Trust**

- Inventory executables/MCP tools observed in Change execution.
- Create tool manifest format with digest, version, publisher/source, capabilities, filesystem/network scope, credential needs.
- Allow trust approval at exact version/digest or publisher-policy level.
- Detect capability drift across versions.
- Block or require approval for unknown tools requesting broad shell/filesystem/network/credential access.

## **P7 - Replay**

- Timeline endpoint and UI.
- Capture causal parent/child links among events and effects.
- Inspect point-in-time repository/resource state from stored before/produced content.
- Export replay bundle with redacted evidence.
- Later add tool-response replay and forked runs.

## 18. MVP Definition

### 18.1 What MVP must do

- Windows-first.
- Works with an existing local Git repository.
- Runs at least Codex/Claude Code/generic shell through a supervised Change.
- Records process tree, command execution, Git state, and repository file effects.
- Captures before-images and can recover supported local changes without erasing unrelated later edits.
- Shows a Change summary with intent, effects, tests/build result, recovery readiness, and persistent host drift summary.
- Provides a replay timeline sufficient to identify the action sequence around a failure.
- Implements one credential-broker integration: GitHub.
- Implements initial Tool Records and trust decisions for externally invoked agent tools.

### 18.2 What MVP must explicitly not promise

- Perfect rollback of arbitrary external APIs.
- Perfect deterministic replay of the outside world.
- Complete OS virtualization.
- Cross-platform parity.
- Proof that a software change is globally correct.
- Automatic attribution of every background machine mutation.
- Compatibility with every agent, IDE, MCP server, package manager, or CI provider.

### 18.3 Killer demo

1. Open a real repository.
2. Run: change run codex
3. Ask Codex to perform a non-trivial refactor.
4. Codex edits files, installs/updates one dependency, runs tests, starts a dev process.
5. Runtime shows a causal Change timeline and Change Review.
6. Add a manual human edit after the agent touched one file.
7. Press Undo.
8. Runtime restores all safe agent effects, preserves the later human edit as a conflict, terminates the agent-owned process tree, and verifies the recovered state.
9. Open Replay and jump directly to the step that introduced the bad dependency or failing test.

## 19. Testing and Quality Strategy

| Test family               | Purpose                                                                        |
|---------------------------|--------------------------------------------------------------------------------|
| Schema/state tests        | Invalid state transitions and malformed effects are rejected                   |
| Filesystem recovery tests | Create/modify/delete/rename, conflict preservation, hard links/symlinks policy |
| Windows path tests        | Case folding, 8.3 aliases, junctions, reparse points, long paths               |
| Process tests             | Child/grandchild process ownership, background servers, forced cleanup         |
| Crash recovery tests      | Daemon/process crash during active Change or recovery                          |
| Git interaction tests     | Dirty working tree, staged changes, branch changes, commits                    |

|                   |                                                                                   |
|-------------------|-----------------------------------------------------------------------------------|
|                   | during Change                                                                     |
| Environment tests | PATH/global package/config drift detection                                        |
| Credential tests  | No raw secret leakage to child environment/logs; expiration and scope enforcement |
| Tool trust tests  | Digest/version/capability drift and denial behavior                               |
| Replay tests      | Stable ordering, causal links, redaction, bundle integrity                        |
| Performance tests | Large repositories, high event rates, long-running agents                         |
| Adversarial tests | Agent attempts to escape supervision or invoke untrusted tools                    |

## 19.1 Correctness invariants

- Never report RECOVERED\_VERIFIED unless all supported planned post-conditions were actually checked.
- Never silently overwrite CURRENT state when it differs from the Change-produced version.
- Never persist raw long-lived credentials in the journal.
- Every effect must resolve to a Change and actor or be explicitly marked unattributed.
- Unrecognized external effects default to unknown semantics, not reversible semantics.
- A tool trust decision is invalidated when the artifact digest or declared capability set changes.

## 20. Security and Threat Model

| Threat                       | Mitigation direction                                                                       |
|------------------------------|--------------------------------------------------------------------------------------------|
| Malicious/compromised agent  | OS-enforced restrictions, scoped broker authority, journal outside child process           |
| Prompt-injected tool call    | Tool trust + capability boundary + credential broker + policy                              |
| Secret exfiltration          | Do not inject durable raw secrets; egress controls later; redact logs                      |
| Agent kills/edits runtime    | Separate daemon/helper permissions; minimal privileged surface; integrity checks           |
| Journal tampering            | Restricted storage + sequence/hash chaining + optional signatures                          |
| Untrusted repository         | Treat repository text as data; native restrictions; no inherited broad credentials         |
| Rollback destroys human work | Three-state before/produced/current comparison and conflict-aware recovery                 |
| Untrusted MCP/plugin         | Tool manifest, signature/digest, capability review, version pinning                        |
| Privileged helper compromise | Tiny auditable native helper; strict IPC schema; no business logic in privileged component |

## 21. Performance and UX Budgets

| Metric                   | Target direction                                                                         |
|--------------------------|------------------------------------------------------------------------------------------|
| CLI start overhead       | < 300 ms before launching common agent command after daemon warm-up                      |
| Journal write            | Asynchronous/batched where safe; should not noticeably stall shell-heavy workloads       |
| File effect visibility   | Near real time for UI, authoritative final reconciliation at Change end                  |
| Change review generation | < 2 s for normal repositories after evidence already collected                           |
| Recovery planning        | < 2 s for hundreds of changed paths; progress for larger scopes                          |
| User approvals           | Rare; focused on scope deviation, irreversible/unknown effects, or untrusted broad tools |

## 22. Feasibility Assessment

| Subsystem              | Feasibility | Difficulty  | Comment                                           |
|------------------------|-------------|-------------|---------------------------------------------------|
| Change model + journal | Very high   | Medium      | Conventional state/event engineering              |
| Process supervision    | Very high   | Medium      | Strong OS primitives                              |
| Local file recovery    | High        | Medium-high | Conflict semantics matter more than copying bytes |

|                                   |                 |             |                                                                     |
|-----------------------------------|-----------------|-------------|---------------------------------------------------------------------|
| Environment provenance            | High            | High        | Must bound scope; avoid pretending to observe entire OS             |
| Assurance engine                  | High            | High        | Adapter-heavy and evidence selection is a product/research problem  |
| GitHub credential broker          | High            | High        | Achievable with careful secret boundaries                           |
| Tool supply chain                 | High            | High        | Manifest ecosystem and trustworthy provenance become long-term moat |
| Trace replay                      | Very high       | Medium-high | Strongly feasible                                                   |
| Controlled/forked replay          | Medium-high     | Very high   | External nondeterminism must be captured/virtualized                |
| Cross-platform native enforcement | High eventually | Extreme     | Do not attempt in first release                                     |

## 22.1 Main feasibility risk

**The principal risk is scope, not computer-science impossibility. A credible Windows-first vertical slice is feasible; a universal transactional runtime across every OS, tool, cloud, and external API is not an MVP.**

# 23. User and Market Viability

## 23.1 Initial user

The first target should be the AI-heavy individual developer or small engineering team already using autonomous coding agents. This user feels the pain immediately, can adopt a local tool without procurement, and can evaluate value in hours rather than months.

## 23.2 Why a user installs it

- They want to let agents run longer without babysitting every command.
- They want to understand large AI-generated changes without reading every low-level action.
- They want a reliable answer to "what did the agent change outside Git?"
- They want to recover bad local work without resetting unrelated human changes.
- They want agent credentials and tools controlled without configuring a containerized development workflow.
- They want debugging evidence when an agent enters a bad sequence of actions.

## 23.3 Adoption requirement

**The product succeeds only if it adds less cognitive overhead than it removes. The default path must require almost no configuration, and ordinary reversible work should not trigger approval spam.**

## 24. Competitive Positioning

| Category                  | What it solves                         | Our position                                                        |
|---------------------------|----------------------------------------|---------------------------------------------------------------------|
| Coding agents             | Generate/modify code and operate tools | Do not replace; govern and observe                                  |
| Agent sandboxes/VMs       | Isolation of execution environment     | Native developer experience + causal Change model                   |
| Git/PR tools              | Repository changes and code review     | Begin before Git diff and continue beyond PR                        |
| CI systems                | Automated validation pipelines         | Consume evidence and tie result to Change lifecycle                 |
| Observability             | Runtime metrics/logs/traces            | Later correlate outcome with Change; do not replace backend         |
| IAM/workload identity     | Identity and authorization primitives  | Apply scoped delegation to agent-driven SDLC Changes                |
| SBOM/provenance standards | Supply-chain evidence formats          | Consume/produce compatible evidence rather than invent new standard |
| LLM tracing               | Prompts/model/tool traces              | Focus on real software effects, machine state, causality, recovery  |

### 24.1 Defensible center

The moat is not a sandbox or a dashboard. It is the accumulated causal model connecting intent, actor authority, real machine effects, tool provenance, verification evidence, replay, and recovery around a persistent Change.

## 25. Post-MVP Roadmap

| Generation | Expansion                                                                                                                               |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| G2         | Better impact graph, contract verification, broader test adapters, GitHub PR/CI continuity, richer Tool Registry                        |
| G3         | Linux/macOS execution adapters, SLSA-compatible artifact provenance, deployment linkage, OpenTelemetry outcome correlation              |
| G4         | Semantic external API adapters, staged actions, compensations, stronger replay, multi-agent concurrency and Change isolation            |
| G5         | Organization control plane, shared trust policies, fleet identity, enterprise audit, historical causal intelligence across repositories |

## 26. Concrete First Implementation Backlog

1. Create monorepo skeleton and coding standards.
2. Define ChangeStatus enum and transition table.
3. Define Change/Actor/Effect schemas.
4. Create SQLite schema and migration framework.
5. Implement append-only journal sequence allocator.

6. Implement local daemon lifecycle and health endpoint.
7. Implement CLI: change new/list/show.
8. Implement CLI: change run <command>.
9. Create Windows process supervisor abstraction.
10. Implement Windows Job Object ownership and termination.
11. Persist process tree events.
12. Implement repository baseline scanner.
13. Implement file before-image store.
14. Implement post-run reconciliation of repository paths.
15. Canonicalize Windows paths and test 8.3/case aliases.
16. Implement ResourceVersion comparison.
17. Implement recovery planner for create/modify/delete.
18. Implement recovery executor and digest verification.
19. Implement conflict state when CURRENT != PRODUCED.
20. Capture Git baseline/after state.
21. Add generic shell agent adapter.
22. Add Codex adapter.
23. Add Claude Code adapter.
24. Implement EnvironmentPassport v1 collector.
25. Implement basic npm/pip dependency diff adapters.
26. Implement generic test command evidence adapter.
27. Add pytest structured result adapter.
28. Add Jest structured result adapter.
29. Build timeline endpoint.
30. Build first Change Review terminal UI or minimal desktop screen.

## 27. G1 Acceptance Criteria

- Running an agent through the runtime creates exactly one Change with durable ID.
- All descendant processes are either attributed or explicitly marked unattributed with reason.
- Repository file changes are visible with before/produced digests.
- A later human edit to an agent-touched file creates a recovery conflict instead of being silently overwritten.
- Undo restores all non-conflicting supported local file effects and verifies hashes.
- Undo terminates Change-owned process tree.
- The journal survives daemon restart and an interrupted active Change is surfaced as interrupted/recovery-required.
- No raw known secret value is written into normal logs/journal in tests.
- Environment Passport can show at least language/toolchain/PATH/global package differences across a Change.
- Replay timeline can locate commands and file mutations around a chosen failure point.
- Typical agent use remains practical without repeated approval prompts.

## 28. Major Risks and Mitigations

| Risk                               | Mitigation                                                                                     |
|------------------------------------|------------------------------------------------------------------------------------------------|
| Scope explosion                    | Windows-first; one Git provider; a few package/test adapters; strict MVP exclusions            |
| Performance overhead               | Event-driven capture + targeted reconciliation; batch journal writes; content-addressed dedupe |
| False sense of rollback safety     | Typed recovery semantics; conflict checks; verified post-conditions; explicit UNKNOWN          |
| Privileged helper complexity       | Minimize native privileged code; strict IPC; aggressive adversarial testing                    |
| Tool ecosystem moves quickly       | Stable manifest schema + adapters; do not bind product identity to one MCP implementation      |
| Agent vendor changes CLI behavior  | Thin adapters around stable generic shell/runtime primitives                                   |
| Environment observation incomplete | Report coverage boundaries; never call unknown state clean                                     |
| Security feature becomes UX burden | Risk-based summaries; safe defaults; approvals only on material boundaries                     |

## 29. Product Success Metrics

| Metric                                            | Why it matters                                            |
|---------------------------------------------------|-----------------------------------------------------------|
| Autonomous minutes per human interruption         | Measures whether safety actually reduces approval fatigue |
| Recovery success rate for supported local effects | Core trust metric                                         |
| Recovery conflict precision                       | Conflicts should represent real later edits, not noise    |
| Unattributed effect rate                          | Measures causal visibility                                |
| Persistent unexplained environment drift          | Measures Docker-free observability quality                |
| Time to identify first bad effect                 | Replay/debugging value                                    |
| Change review time vs raw diff review             | Assurance UX value                                        |

|                                             |                              |
|---------------------------------------------|------------------------------|
| Percentage of credential use through broker | Security adoption            |
| Unknown/untrusted tool invocation rate      | Tool supply-chain visibility |

## 30. Target Change Passport

CHANGE C-1048

=====

### INTENT

Rotating refresh-token implementation

### OWNER / ACTORS

Human: Shlok

Agent: Codex AG-182

Processes: 27 attributed / 0 unknown

### AUTHORITY

GitHub: repository read + branch push

Production: none

Durable secrets exposed to agent: 0

### TOOLS

Codex CLI: approved

GitHub broker: approved

Unknown MCP/skills: 0

### SOURCE EFFECTS

17 files modified

2 files created

1 file deleted

1 dependency upgraded

### ENVIRONMENT

Node 24.7 / npm 11.6 / Python 3.13.7

Persistent host drift: 0

### ASSURANCE

Intent criteria: 6/6 covered

Unit tests: 438/438 pass

Integration tests: pass

Build: pass

Security checks: pass

Unverified: concurrent token rotation stress case

### RECOVERY

Files: exact

Processes: exact

Dependency state: exact

Host environment: no residual drift

External effects: none

### REPLAY

Trace complete: yes

Captured checkpoints: 5

### STATUS

REVIEW\_READY

=====

## 31. External Technical Foundations and References

The architecture should leverage established primitives and standards rather than invent proprietary equivalents. The following areas are particularly relevant during implementation:

| Foundation                                               | How it informs this project                                                               |
|----------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Windows Job Objects                                      | Process-tree supervision, accounting, and termination for Windows-first runtime.          |
| Windows restricted tokens / sandbox process capabilities | OS-enforced reduction of child-process authority.                                         |
| Linux Landlock                                           | Future native Linux restriction backend without requiring Docker as the user abstraction. |
| macOS Endpoint Security                                  | Future macOS process/file event observation and enforcement.                              |
| OAuth 2.0 Token Exchange patterns                        | Subject/actor delegation model for agents acting on behalf of humans.                     |
| SPIFFE/SPIRE workload identity concepts                  | Workload identity and runtime-attested identity principles.                               |
| SLSA build provenance                                    | Future artifact provenance and supply-chain evidence.                                     |
| OpenTelemetry                                            | Future production correlation of deployed service version with Change outcome.            |
| OWASP Agentic / Agent Control guidance                   | Threat model for tool misuse, identity, runtime controls, and traceability.               |
| Git / CI providers                                       | Evidence sources, not systems to replace.                                                 |

### 31.1 Selected public sources

- OpenAI - Building the Codex Windows sandbox: <https://openai.com/index/building-codex-windows-sandbox/>
- Microsoft - Windows Job Objects: <https://learn.microsoft.com/windows/win32/procthread/job-objects>
- Linux kernel - Landlock documentation: <https://docs.kernel.org/userspace-api/landlock.html>
- Apple - Endpoint Security documentation: <https://developer.apple.com/documentation/endpointsecurity>
- RFC 8693 - OAuth 2.0 Token Exchange: <https://www.rfc-editor.org/rfc/rfc8693>
- SPIFFE specifications: <https://spiffe.io/docs/latest/spiffe-specs/>
- SLSA Build Provenance: <https://slsa.dev/spec/v1.2/build-provenance>
- OpenTelemetry semantic conventions: <https://opentelemetry.io/docs/specs/semconv/>
- OWASP GenAI Security Project: <https://genai.owasp.org/>
- DORA research and insights: <https://dora.dev/>

## 32. Final Recommendation

Proceed with the project, but preserve the narrow center established in this proposal. The product should not become another coding agent, generic security dashboard, generic IAM platform, CI provider, or Docker replacement. Its unique value is the causal and verifiable lifecycle of a software Change across agent execution and the native developer environment.

**Implementation rule for the first release: build the smallest system that can run an existing agent on a real repository, attribute what it did, show what changed outside Git, verify relevant local outcomes, replay the failure sequence, and recover supported effects without destroying unrelated work.**

If that vertical slice is reliable, the architecture can grow naturally into CI continuity, artifact provenance, production outcome correlation, additional OS backends, semantic external effect adapters, and enterprise governance without changing the foundational object model.